



دانشگاه تهران
پردیس دانشکدگان فنی
دانشکده برق و کامپیوتر



سیگنال و سیستم

تمرین کامپیوتری شماره ۱

استاد: دکتر سعید اخوان

نام و نام خانوادگی

سید علی رضاعی

شماره دانشجویی

۸۱۰۱۰۳۴۳۲

بهار ۱۴۰۵

بخش اول:

```
1 % part 1_1
2 - a = 10;
3 - b = 2.5 * 10^23;
4 - c = 2 + 3*1i;
5 - d = exp(1j*2*pi/3);
```

در قسمت اول ابتدا متغیرهای مدنظر تعریف شده اند. نکته ای که میتوان در این قسمت به آن اشاره کرد این است که زمانی که میخواهیم یک عدد مختلط را تعریف کنیم به جای استفاده از i خالی به $1i$ قبل از آن اضافه میکنیم برای استفاده از j هم به همین صورت است. و در نهایت برای عدد نپیر به توان i چیزی از تابع $\exp()$ استفاده میکنیم.

```
7 %part 1_2
8 - aVec = [3.14,15,9,26];
9 - bVec = [2.71;8;28;182];
10 - cVec = 5:-0.2:-5;
11 - dVec = logspace(0,1,100);
12 - eVec = 'Hello';
```

در قسمت دوم برای تعریف یک بردار هم میتوان از اسپیس در بین عناصر موجود در داخل آن استفاده کرد و هم از ، استفاده کرد. برای اینکه در سطر بعدی ماتریس بنویسیم باید از ؛ استفاده کرد.

در خط ۱۰ ام به برداری از ۵ تا منفی ۵ با فاصله ۰.۲- ایجاد میکنیم و آن را در داخل $cVec$ میریزیم.

در خط ۱۱ ام تابعی استفاده شده است که در توان ۱۰ ها یک دنباله ایجاد میکند که در المان اول نقطه شروع و در المان دوم نقطه پایان و المان سوم تعداد نقاطی که در بین این دو عدد درج میکند را مشخص میکند. و در نهایت $eVec$ یک استرینگ است در واقع برداری از کاراکتر هاست.

```
14 %part 1_3
15 - aMat = 2 * ones(9);
16 - vb = [1,2,3,4,5,4,3,2,1];
17 - bMat = diag(vb);
18 - vc = 1:100;
19 - cMat = reshape(vc,[10,10]);
20 - dMat = NaN(3,4);
21 - eMat = [13,-1,5;-22,10,-87];
22 - fMat1 = floor(-3 + (3+3)*rand(5,3));
23 - fMat2 = randi([-3,3],5,3);
```

تابع $ones$ تابعی است که تمام المان های آن یک برمیگرداند و میتوان ابعاد آن را مشخص کرد. زمانی که یک عدد تنها وارد میکنیم به معنی این است که یک ماتریس مربعی از آن دیمانسیون برمیگرداند.

تابع $diag$ دوتا کار میتونه بکنه اول اینکه اگه روی یه ماتریس صدای بزنیم قطر آن را می دهد و اگر روی یک بردار آن را صدا بزنیم آن را به یک ماتریسی که قطر آن اعداد روی آن بردار بوده تبدیل میکند و سایر آرایه های ماتریس را صفر میکند.

تابع `reshape` به بردار رو ابعادش رو به اون ابعادی که می‌خواهیم تبدیل میکند. لازم به ذکر است که ترتیب پرکردن آن به این صورت است که ابتدا ستون‌ها را ثابت و سطر‌ها را تغییر می‌دهد و بعد از تمام شدن سطر‌ها به ستون بعدی می‌رود و دوباره این کار را تکرار میکند.

در خط ۲۲ از آنجایی که تابع `rand` اعداد بین ۰ و ۱ می‌دهد برای اینکه به محدوده مدنظر ما برسه تکنیک زیر را اجرا می‌کنیم: (بر گرفته از `help` متلب)

In general, you can generate N random numbers in the interval [a,b] with the formula $r = a + (b-a) \cdot \text{rand}(N,1)$.

و تابع `floor` همان براکت است.

تابع `randi` در واقع `random integer` است در ورودی اولی محدوده را می‌گیرد و در دو مقادیر دیگه دیمانسیون را دریافت می‌کند.

```
25 %part 1_4
26 - x = 1/(1+exp(-(a-15)/6));
27 - y = (sqrt(a) + nthroot(b,21))^(pi);
28 - z = log(real((c+d)*(c-d))*sin(a*pi/3))/c*conj(c);
```

در این قسمت از کد موارد خواسته شده انجام شده است و تفاوت تابع `nthroot` و اینکه با $^{\wedge}$ ریشه‌ها را حساب کنیم این است که `nthroot` مقدار حقیقی را بر می‌گرداند.

```
30 %part 1_5
31 - xMat = (aVec * bVec)*aMat^2;
32 - yMat = bVec * aVec;
33 - zMat = det(cMat) * ((aMat * bMat) .') ;
```

در ضرب ماتریس‌ها باید دقت شود که دیمانسیون‌ها درست باشد و عملگر `'` `transpose` ماتریس را بر می‌گرداند و نحوه استفاده آن در خط ۳۳ نشان داده شده است.

```
35 %part 1_6
36 - cSum = sum(cMat,1);
37 - eMean = mean(eMat,2);
38 - eMat = [1,1,1;-22,10,-87];
39 - cSub = cMat(2:9,2:9);
```

```
41 - lin = 1:20;
42 - negEven=-lin(2:2:20);
43 - for i = 1:20
44 -     if mod(i,2) == 0
45 -         lin(i) = negEven(i/2);
46 -     end
47 - end
48 - lin;
49
50 - r = rand(1,5);
51 - r(find(z<0.5)) = 0;
```

زمانی که تابع `mean`، `sum` و ... را استفاده میکنیم اگر ورودی دوم را خالی بگذاریم به معنی این است که جمع را روی دیمانسیون اولی انجام میدهد. اگر بخواهیم مانند خط ۳۷ آن را به صورت یک ستون به ما بدهد باید تابع را روی دیمانسیون دومی میانگین بگیریم.

برای اینکه بخواهیم مقدار `lin` یا `یه برداری` از `یه متغیر` را به دست آوریم اسم آن متغیر را نوشته و در داخل پرانتز جلو آن اون قسمت را صدا میزنیم همانند خط ۳۹

از خط ۴۱ تا ۴۸ مربوط به قسمتی است که میخواهیم قسمت های زوج `یه بردار` رو قرینه کنیم. در ابتدا بردار کلی را طبق گفته ی دستور کار تعریف میکنیم سپس یک متغیر دیگر به نام `negEvent` تعریف کرده و قرینه قسمت های زوج را در داخل آن میریزیم در حلقه فور از ۱ تا ۲۰ چک میکنیم اگر زوج بود بره توی `lin` و اون المان `i` رو به اون قرینه روح تغییر بده و در نهایت داخل `lin` بریزه و در نهایت `lin` را صدا زدیم.

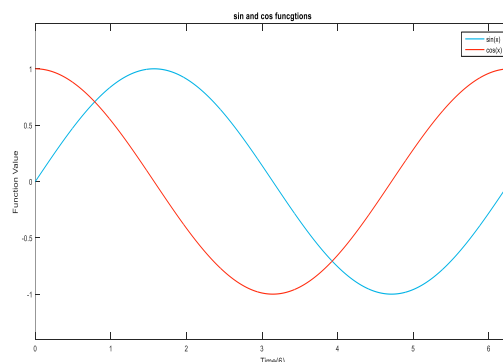
در قسمت بعدی `find` میاد شماره اون المان رو برمیگردونه پس توی خود `r` میگیم برو پیدا کن جا هایی که `r` کمتر از نیم شده رو و شماره اش رو برگردون بعد میگیم این شماره ها رو برو صفر اشون کن.

```
1 - figure
2
3 - t = 0:0.01:2*pi;
4 - plot(t,sin(t), 'Color', [0,0.7,0.9]);
5
6 - hold on;
7
8 - plot(t,cos(t), 'Color', [255,44,15]/255);
9
10 - xlabel('Time(6)');
11 - ylabel('Function Value');
12 - title('sin and cos functions');
13 - legend('sin(x)', 'cos(x)');
14 - xlim([0,2*pi]);
15 - ylim([-1.4,1.4]);
```

توی قسمت پلات کردن چند تا نکته هست اول اینکه هر کدوم از فیگور ها `یه صفحه` رو باز میکنه در ابتدا اینکارو کردیم اگه مثلا بخواهیم چندتا پلات رسم کنیم توی صفحات مختلف میتونیم از فیگور های با شماره ۱ تا `n` استفاده کنیم.

توی رنگ پلاتی که میخواهیم باید همه اش بین صفر تا یک باشه برای همین من توی دومی همه رو تقسیم بر ۲۵۵ کردم و در نهایت آن ها را ترسیم کردیم.

خروجی قسمت ۱-۷ به شکل زیر است:



بخش دوم:

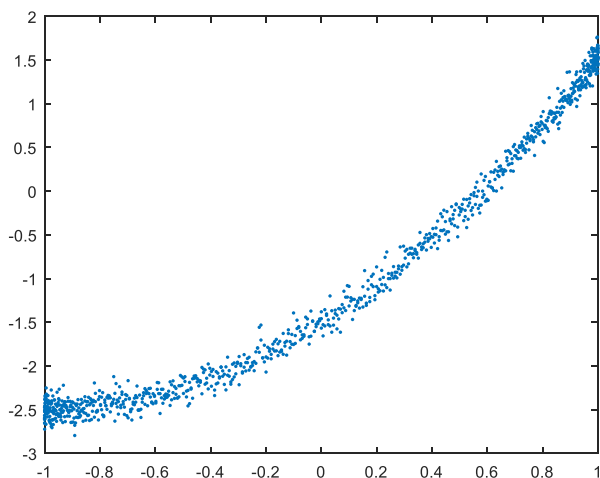
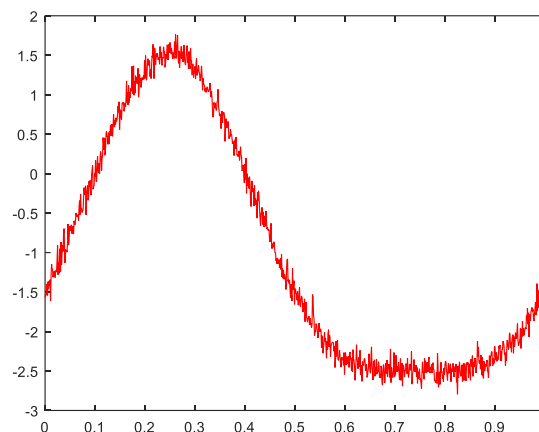
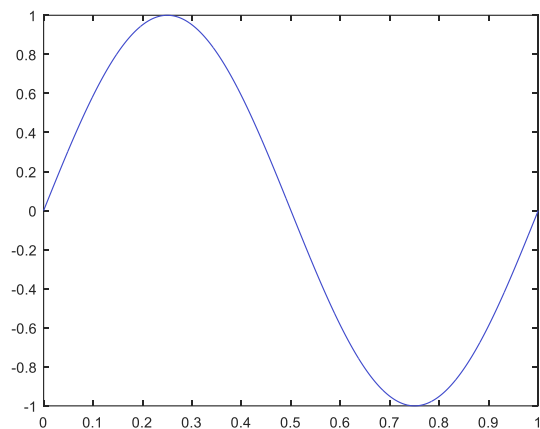
```
1 - load('p2.mat');
2
3 - figure(1)
4 - plot(t,x,'color' , [63,74,204]/255);
5
6 - figure(2)
7 - plot(t,y,'color' , [1,0,0]);
8
9 - figure(3)
10 - plot(x,y,'.');
11
```

خط ۱: یک فایل را لود کردیم، بعد از لود شدن متغیرهایی که در داخل آن بود در workspace نمایان شد.

خط ۳،۶،۹: فیگورهای ۱ و ۲ و ۳ یعنی ۳ پنجره متفاوت را برای نمایش باز میکند.

در خط ۴ ایکس بر حسب تی و در خط ۷، وی بر حسب تی پلات شده است و درباره رنگ آن در بخش یک توضیح کامل داده شده است. در خط ۱۰ نیز '.' به معنی این است که به صورت نقطه ای نمایش بدهد.

نتایج آن به شکل زیر است.



کد بخش ۴ به صورت زیر است که به توضیح آن خواهیم پرداخت

```

1 function [ao, bo, co] = p2_4(x, y)
2
3     a = -5:0.01:5;
4     b = -5:0.01:5;
5     g = -5:0.01:5;
6
7     m = inf;
8
9
10    for k = 1:length(g)
11        for j = 1:length(b)
12            for i = 1:length(a)
13                f = sum((y-a(i)*(x.^2)-b(j)*x-g(k)).^2);
14                if f < m
15                    m = f;
16                    i_m = i;
17                    j_m = j;
18                    k_m = k;
19                end
20            end
21        end
22    end
23
24    ao = a(i_m);
25    bo = b(j_m);
26    co = g(k_m);
27
28 end

```

منطق کلی کد: به بازه ای رو در نظر میگیریم که احتمال می دهیم ضرایب مجهول در داخل آن باشند(بازه را نباید خیلی بزرگ هم در نظر گرفت زیرا فرآیند بررسی کد زیاد طول خواهد کشید). بعد روی آن ۳ تا حله فور در نظر میگیریم و سعی میکنیم که مینیموم را به دست آوریم برای این منظور در داخل فور آخری میگوییم که اگر مقدار تابعمون که تابعی از a, b, c است از مقدار m کمتر شد بیا f رو برابر m بذار بعد ضرایب k, j, i رو بذار برابر مینیمومشون یعنی k_m, j_m, i_m توجه شود که برای اینکه مقدار اولی ام به عدد بزرگ باشه از inf استفاده شده است. (خط ۷)

خط ۱: تعریف تابع که دو ورودی میگیرد x, y و خروجی های آن ao, bo, co هستند o رو برای اینکه اوت پوت هستن بهشون اضافه کردم

خط ۳ تا ۵: ۳ تا وکتور تعریف کردیم که هر کدام بازه ای که میخواهیم ضرایب مجهول را در داخل آن باشد در نظر میگیریم خطوط بعدی در منطق کنلی توضیح داده شد

در خط ۱۳ نکته ای که هست این است که در متلب برای اینکه مثلاً x را هر المان رو به توان ۲ برسونیم باید توی ترانهاده اش ضرب بشه که هر المان به توان ۲ برسه برای همین از $.^2$ استفاده کردم

و در خط ۲۴ تا ۲۶ میگیریم که برو از a اون المانی رو بیار و به عنوان خروجی بده که شماره اش شد i_m برای ۲ خط بعدیش

```
>> [a,b,c] = p2_4(x,y)
```

```
a =
```

```
1.0100
```

```
b =
```

```
2.0100
```

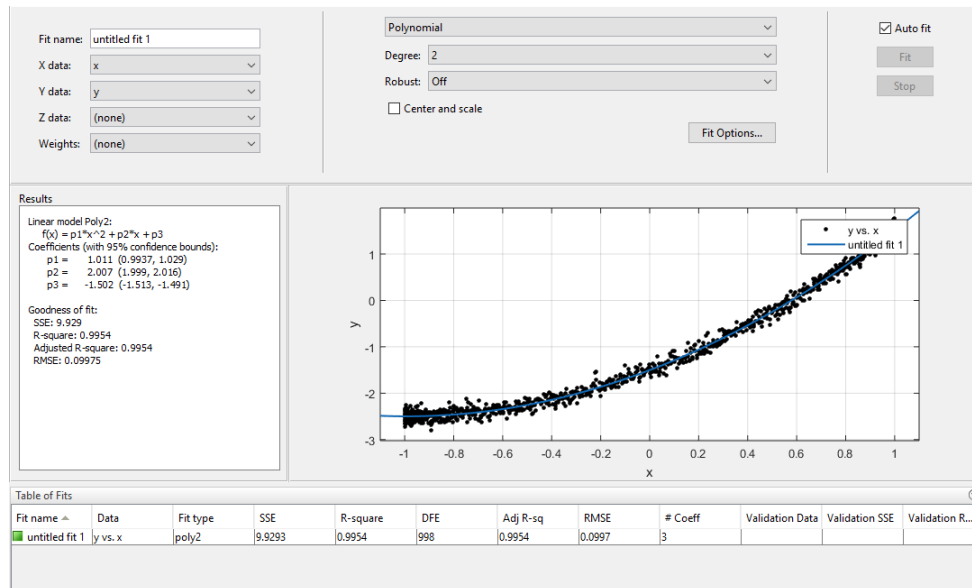
```
c =
```

```
-1.5000
```

هم همین طوریه

خروجی آن به شکل روبه رو خواهد بود:

در تمرین ۵_۲ با استفاده از کرو فیتینگ این کار انجام دادیم و خروجی آن کاملاً نزدیک قسمت قبل است و این یعنی درست کار میکند همه چی و خروجی قسمت ۵_۲ به شکل زیر است:



بخش سوم:

```

1 - ts = 1e-9;
2 - T = 1e-5;
3 - tau = 1e-6;
4
5 - signalWidth = tau/ts;
6
7
8 - t = 0:ts:T;
9 - sent_signal = zeros(1,length(t));
10 - sent_signal(1:signalWidth) = 1;
11
12 - figure(1);
13 - plot(t, sent_signal, 'color', [101,11,200]/255,'LineWidth',1.3);
14
15 - title('sent signal');
16 - xlabel('time (s)');
17 - ylabel('amplitude');
18
19 - xlim([0,T]);
20 - ylim([0, 1.5]);
21 - grid on;
22 - hold on;
23
24 - trs = 3e-6; %the time that our detector receives the signal
25 - alfa = 0.5;
26 - res_signal = zeros(1,length(t));
27 - res_signal( trs/ts : trs/ts + signalWidth) = alfa;
28
29 - plot(t, res_signal,':', 'color', [0,128,128]/255,'LineWidth',2);
30
31 - conv = zeros(1,10001);
32 - for i = 1:length(t)-signalWidth
33 -     conv(i) = sum((sent_signal(:) .* res_signal(:)));
34 -     sent_signal = zeros(1,length(t));
35 -     sent_signal(i:i+signalWidth-1) = 1;
36 - end
37
38 - figure(2);
39 - plot(t,conv);
40 - grid on;

```

توضیح کد

خط ۱ تا ۳: مغییر های دستور کار تعریف شده اند.

خط ۵: برای راحتی استفاده در طول کد نویسی یه متغییر دیگه تعریف کردم به نام signalWidth که عرض پالس ها رو

نشان میده که با تقسیم زمان طول سیگنال به بازه های نمونه گیری به دست خواهد آمد.

در ادامه خطوط ۸ تا ۹ سیگنال ارسال شده را شبیه سازی کردم برای این منظور اول یه بردار به طول اندازه t همه رو صفر

میکنیم بعد از المان اولی تا اندازه ی سیگنال رو مقدار دهی میکنیم به ۱ بدین ترتیب سیگنال پالس را ساختیم.

سپس ان را رسم میکنیم در رسم زخامت سیگنال را مقداری بیشتر کردم برای بهتر دیده شدن و grod on هم دستور خوبی

که توی این بخش یاد گرفتم و چارت رو خط کشی میکنه خیلی به بهتر دیده شدن کمک میکنه.

تا اینجا بخش ۱-۳ تموم شد

در ادامه با محاسبه به مقدار trs به معنی زمانی که سیگنال رو دریافت میکنیم خواهیم رسید که محاسبات به شکل زیر است:

$$trs = \frac{2R}{C}, R = 450, C = 3 \times 10^8 \rightarrow trs = 3 \times 10^{-6} = 3e - 6$$

و در ادامه همانند قسمت قبل در تولید سیگنال عمل میکنیم. توجه شود که trs/ts میشه شماره اون المانی که از اونجا

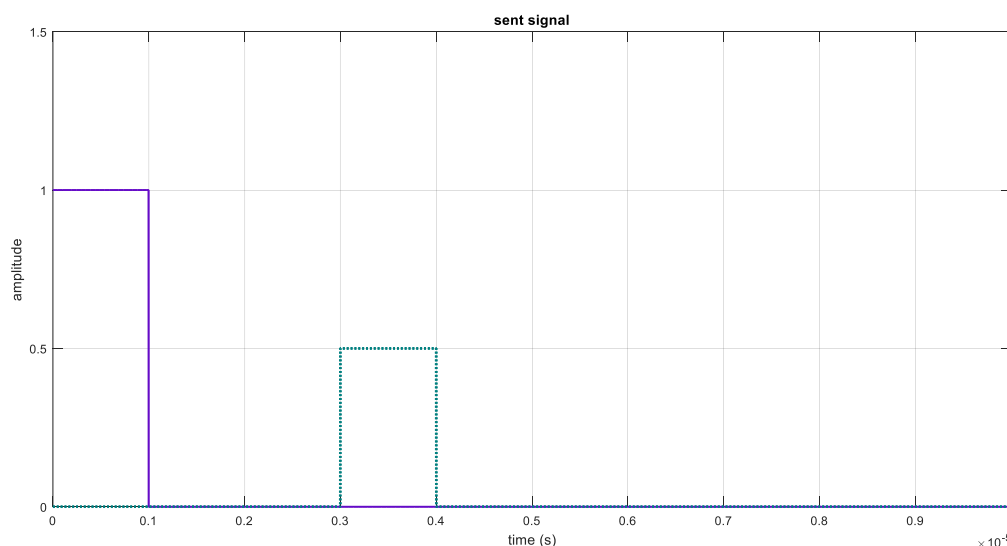
سیگنال شروع شده است این مقدار را اگر به اندازه ی signalWidth جلو ببریم رو مقدار دهی کنیم به آلفا که طبق دستور کار

نیم است سیگنال را درست کردیم و در ادامه آن را پلات میکنیم با استفاده از ':' نحوه نمایش آن را متفاوت کردم و در ادامه برای

بهتر دیده شدن زخامت ان را افزایش دادم.

پایان بخش ۲-۳

خروجی آن به شکل زیر است.



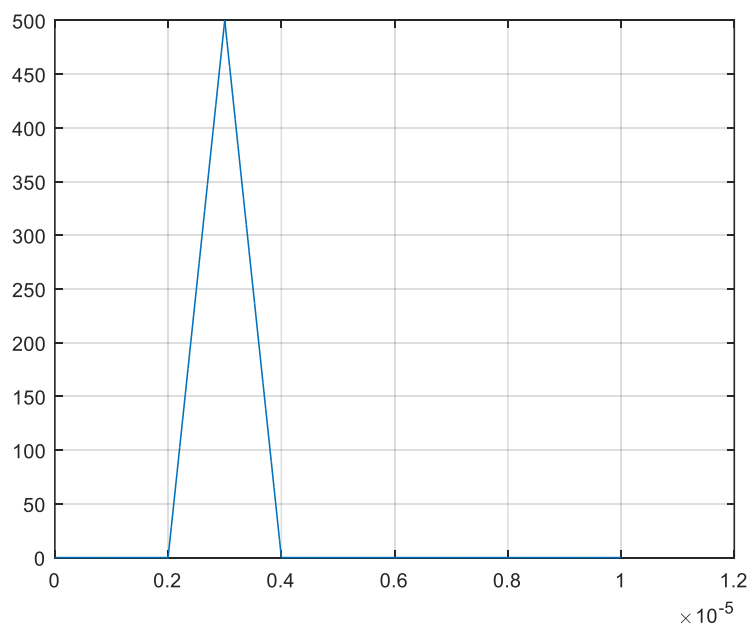
در ادامه میخواهیم که از این دو سیگنال کانولوشن بگیریم نکته ی که فهمیدم توی این تمرین این بود که برای سریع شدن

کد خیلی خوبه که همون اول conv رو بهش بگیم که یه برداریه است مثلاً تمام المان های آن صفر و در ادامه توی حلقه فور

مقدار المان های آن را تغییر بدهیم اگر این کارا نکنیم یعنی هر دفعه توی حلقه فور بخواد بره به اندازه بردار اضافه کنه خیلی زمان

بیشتری میگیره این اتفاق به دلیل این است که میره هر دفعه توی حافظه یه جای جدید پیدا میکنه ولی اگه قبلش بهش بگیم یه جای کامل رو اختصاص میده به متغیرمون و خیلی خیلی زمان بهینه میشه این اتفاق توی پروژه های بزرگ تر بسیار تعیین کننده میشه.

توی حلقه فور میگیریم که هر دفعه بیا به ازای i از ۱ تا طول کلی منهای طول سیگنال (چون میخوایم سیگنال را انتقال بدیم تا این جا جلو تر نمیشه رفت) بیا همه المان های توی این دوتا سیگنال رو در هم ضرب نقطه ای کن یعنی هر کدومو توی متناظرش ضرب کن و بعدش جمعش کن این میشه المان i ام conv بعد سیگنال رو توی خط ۳۴ و ۳۵ یه پله انتقال میدیم به جلو بعد از پایان این حلقه شکل کرولیشن این دوتا سیگنال به دست میاد که در نقطه ی max یعنی اینجا رسیده سیگنال بهمون یعنی در زمان 0.3×10^{-5} سیگنال به دستمون رسیده که با توجه به فرمول $\frac{2R}{c} = 0.3 \times 10^{-5}$ فاصله ان را میتوان محاسبه کنیم.



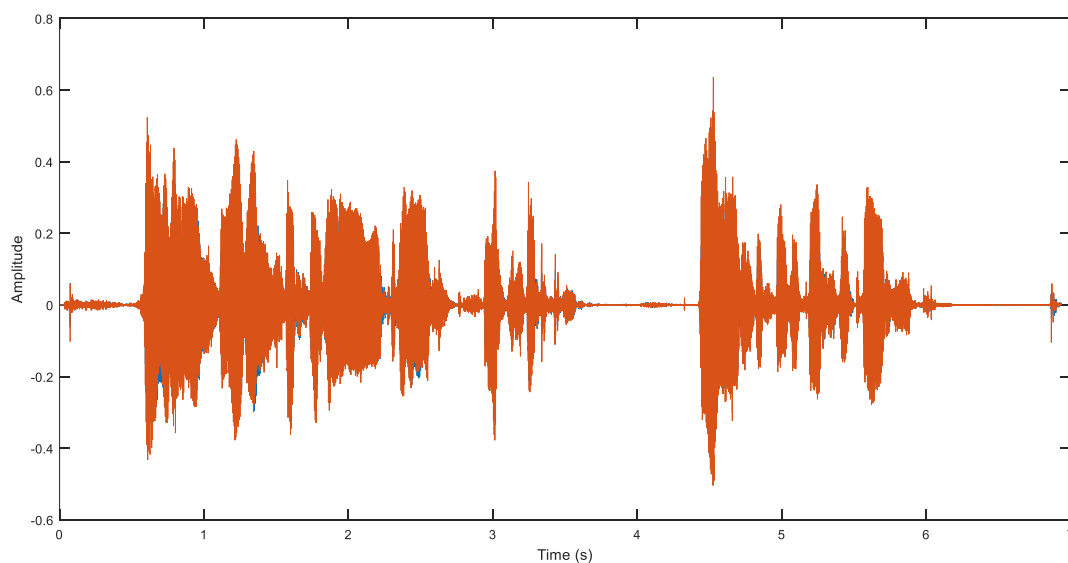
بخش چهارم:

```
1 - [x,Fs] = audioread('myVoice.wav');
2
3 - lx = length(x);
4 - lspan = 1:lx;
5 - t = lspan/Fs;
6
7 - hf = figure();
8 - plot(t, x)
9 - xlabel('Time (s)');
10 - ylabel('Amplitude');
11
12 - sound(x,Fs);
13
14 - audiowrite('x.wav',x,Fs);
```

خط ۱: با استفاده از `audioread` در داخل آرگومان اش باید اسم و پسوند فایل مورد نظر باشد. (توجه شود که فایل باید در داخل پوشه باشد.) و دو تا خروجی از این دریافت میکنیم اولی که `x` همون صداس که دوتا حالت داره اگر مونو ذخیره شده باشه این یه آرایه با یه ستون است اگر استریو ضبط کرده باشید این دوتا ستون داره هر کدوم از اون ستون ها مربوط به یک اسپیکر است و در کل دارای تعداد کلی نمونه برداری شده است. و `Fs` نشان دهنده میدهد که در هر ثانیه، چند نمونه از سیگنال پیوسته گرفته شده است. در این جا `Fs` برابر ۴۸۰۰۰ شده است.

در خط ۳ تا ۵: برای رسم سیگنال بر حسب زمان فرکانس را به زمان تبدیل کردیم برای این کار برداری به اندازه ی ۱ تا اندازه ی `x` درست کردیم و سپس تقسیم بر فرکانس کردیم تا در هر لحظه زمان آن در بیاید. که بتوانیم در ادامه بر حسب زمان آن را رسم کنیم.

و در ادامه لیبل گذاری انجام داده شده است و پلات رسم شده است. شکل آن را میتوانید در زیر مشاهده کنید. (اینکه یه جایی نقاط آبی مشاهده میشود برای این است که فایل استریو است و همان طور که توضیح داده شد یه ماتریس ۲ بعدی است.)



در ادامه آن را با دستور `sound` پخش کردم و در نهایت با استفاده از دستور `audiowrite` آن را با نام فایل `x` و نرخ نمونه برداری `Fs` در داخل همون فولدر ذخیره کردم.

```
1 - [x,Fs] = audioread('x.wav');
2 - speed = 1.5;
3 - p4_3(x,speed)

1 - function p4_3(x,speed)
2 -     sound(x,speed*48000);
3 - end
```

در تمرین ۳_۴ در فایل `part4mainCode2` (تصویر بالایی) یه سرعت تست و یه فایل رو گذاشتم و تابعی که در تصویر پایین نوشتم رو صدا زدم توی این تابع کاری که میکنیم با ضرب کردن `speed` در فرکانسی که نمونه برداری میکرد (۴۸۰۰۰) سرعت آن را تغییر میدهیم. چون در واقع میاد صدا رو هر ثانیه یه تعداد بیشتری ازش نمونه برداری میکنه در سرعت های بیشتر از یک و باعث میشه که تند تر شنیده بشه.

و شعری هم که خواندم بیت زیر است :

آینه چون نقش تو بنمود راست / خود شکن، آینه شکستن خطاست